A closed-loop controller for Kaggriculture

Technical report - prepared 2026-11-09 - not peer-reviewed

Author: Ryo Hasegawa

Abstract

Kaggriculture is a two-player farming simulation in which an agent must allocate labor, land, seeds, animals, feed, fertilizer, and market actions across a 30-day, 720-turn season. This report documents a retained single-file closed-loop planner. Instead of replaying one fixed action sequence, the controller reads the observed farm and market state each turn, builds a job list, assigns the farmer and hands by urgency and distance, and emits market orders from current inventory and price conditions. The official final leaderboard record for the associated team is rank 76 of 10,148 with a displayed score of 4,856.5 and a Silver medal. The exact source hash of the final hosted submission was not retained; consequently, this report makes a documented source-lineage claim rather than claiming byte identity between the released `main.py` and submission `56719108`.

1. Competition and evaluation setting

The official Kaggriculture overview describes a turn-based farming simulation with two farms, 24 turns per day, and 30 days per season. The objective is to finish with more coins in the bank. The environment combines crop and animal production, hiring, land expansion, town demand, and inventory-sensitive market prices. The official leaderboard reports the official final result; it is not equivalent to a local self-play or held-out battery.

The final record was captured from Kaggle's leaderboard services on 2026-11-09 local time (2026-11-10 02:51:29 UTC):

- team: Ryo Hasegawa, one member;
- final submission ID: `56719108`;
- official final rank: `76 / 10,148`;
- displayed score: `4,856.5`;
- medal: `SILVER`;
- recorded submissions: `2`.

The response field used by Kaggle's internal service was named `privateLeaderboard`; Kaggriculture's simulation rules describe a single official final leaderboard. The structured evidence is kept in `results/official_final.json`.

2. Controller design

2.1 Observation and state

The controller's `World` representation is built from the current observation, including the farm tiles, units, inventories, shop draw, town state, and step clock (`main.py:325-469`). Persistent per-player state tracks roles, targets, daily progress, and market history. This permits decisions to change when a shop appears, a tile becomes a weed, a herd expands, or observed market prices move.

2.2 Production and labor planning

The source encodes the competition's crop, animal, shop, land, and clock constants (`main.py:38-87`). At a daily boundary it assigns field roles and updates production targets (`main.py:485-927`, `main.py:970-1004`). Each turn it derives jobs from actual plants, animals, seeds, shed stock, and structure needs (`main.py:1066-1407`). Jobs include watering, feeding, caring, collecting, harvesting, fertilizing, planting, building, digging, and carrying goods.

Assignment is deliberately operational rather than purely economic. Priorities include production deadlines and animal/feed urgency, with walking distance and unit stickiness used to reduce avoidable travel (`main.py:1440-1563`, `main.py:1826-2091`). The final action path returns separate farmer, hand, and market-order lists (`main.py:4075-4128`). A defensive wrapper returns pass actions if an unexpected observation or optional engine import fails (`main.py:4207-4220`).

2.3 Market control

The market model is inventory-dependent: prices are computed from the current shared inventory, and `units_above_price` simulates one-unit sales until a floor is reached (`main.py:90-128`). The controller therefore reserves feed wheat, liquidates surplus, gives premium goods an ordering weight, and uses product-specific floors rather than assuming a fixed price (`main.py:3102-3128`, `main.py:3220-3580`). This is a resource-allocation policy: production volume is heuristically balanced against labor, shed capacity, town drain, and the risk of depressing the shared price.

2.4 Optional research branches

The retained file contains route and replay/tape branches for provenance, but the relevant switches default to off (`main.py:1519-1538`, `main.py:2094-2107`). The public package excludes replay libraries, oracle schedules, private game traces, and development logs. The line selected in the development record used a replayed opening keyed to the shop draw followed by the planner; the private opening library is not shipped, so the released default runs the planner from step 0. Whether submission `56719108` embedded that opening is not determinable from retained artifacts.

3. Research decisions and negative findings

The dated development record distinguishes the selected direction from rejected alternatives. These findings are included to document rejected alternatives rather than presenting only the final score.

First, a market-order tape was compared on `n=4` leader games. It lost approximately 107k, 84k, 66k, and 89k coins, versus baseline losses of 4.5k, 3.6k, 1.8k, and 17k. The recorded orders were conditioned on the reference farm's inventory; produce in the new farm then remained unsold. The tape branch was left disabled and market orders stayed state-dependent.

Second, a full-game action-history tree replay reached 31/192 in the recorded local battery and produced heavy losses on several opponents. Pure full-game replay was rejected in favor of the replayed-opening-plus-planner line. These were local proxy evaluations, not hosted leaderboard measurements, and the private opening library is not included in this release.

4. Official result and provenance

The release records the official result separately from source provenance. The released `main.py` has SHA-256:

`5a6668f4cd7ded9b9865f1334881e97dda0a3d2545a1fe31fd1d40aedddcefd6`

The hosted final result is identified by submission ID `56719108`, but the inspected local artifacts do not contain that submission's source hash. The appropriate claim is therefore:

The released file is a retained closed-loop planner lineage associated with the Kaggriculture work, while rank 76/10,148 and Silver are independently verified official final results for the team.

This distinction prevents a local source snapshot from being misrepresented as the exact code that earned the hosted result.

5. Reproducibility and scope

The package is intentionally small:

1. Run `py -3 -m py_compile main.py` or `python3 -m py_compile main.py`.
2. Run `py -3 tests/smoke_test.py` or `python3 tests/smoke_test.py`.
3. For optional engine evaluation, install `kaggle-environments==1.32.7` from `requirements.txt` on a compatible Python version, then load `main.py` through the official Kaggle Environments API.

The smoke test checks importability without credentials, market arithmetic, fallback action shape, and the retained source hash. A four-step optional engine loader smoke using Python 3.11.15 ended with both agents `DONE` and rewards `150.0` and `150.0`. A separate step-0 check returned `BUILD_PASTURE` and hiring/purchase orders with no stderr, confirming a non-fallback planner decision. These checks cover startup behavior, not a complete game or competition result. The archive does not include data or replay assets required by disabled experimental branches.

6. Limitations

- The final leaderboard score is official, but the exact final source artifact is unresolved.
- Local batteries are proxies whose opponent sets, seeds, and selection history are not equivalent to the official final evaluation.
- The released source mirrors public environment constants and synchronizes against `kaggle-environments` by default, falling back to its built-in constants when the package is absent; it does not vendor the Kaggle engine.
- `main.py` is the retained user-owned source snapshot, © 2026 Ryo Hasegawa. All rights reserved; this archive makes no new license grant. See `THIRD_PARTY_NOTICES.md`.
- This is a technical report prepared for archival and portfolio use, not peer-reviewed research.

References

1. Kaggle. “Kaggriculture.” Official competition overview and rules, accessed 2026-11-09. <https://www.kaggle.com/competitions/kaggriculture/overview>
2. Kaggle. “Kaggriculture leaderboard,” accessed 2026-11-09. <https://www.kaggle.com/competitions/kaggriculture/leaderboard>
3. Kaggle. `kaggle-environments`, Apache-2.0 licensed runtime, accessed 2026-11-09. <https://github.com/kaggle/kaggle-environments>
4. Released source and evidence files: `main.py`, `results/results.md`, `source_manifest.json`. These are archival artifacts, not external peer-reviewed sources.